hhu

hhu
Heinrich Heine
Universität Düsseldorf

Isolation and Protection in Operating Systems

3. System Calls in x86-64

Prof. Dr. Michael Schöttner
Dr. Fabian Ruhland

State & Next Steps for HeineOS

Heinrich Heine
Universität Düsseldorf

- Right now all user mode threads are running in ring 3 but still can call all kernel functions directly
  - The CPU triggers an General Protection Fault (GPF) if a privileged instruction is used inside one of the called kernel function or in any app
  - Usually, kernel code is protected from direct execution in ring 3 via paging (will be done later)

- We now want to implement *System Calls*, which export selected kernel functions for indirect and controlled execution.
  - The CPU will switch from ring 3 to ring 0 during a system call

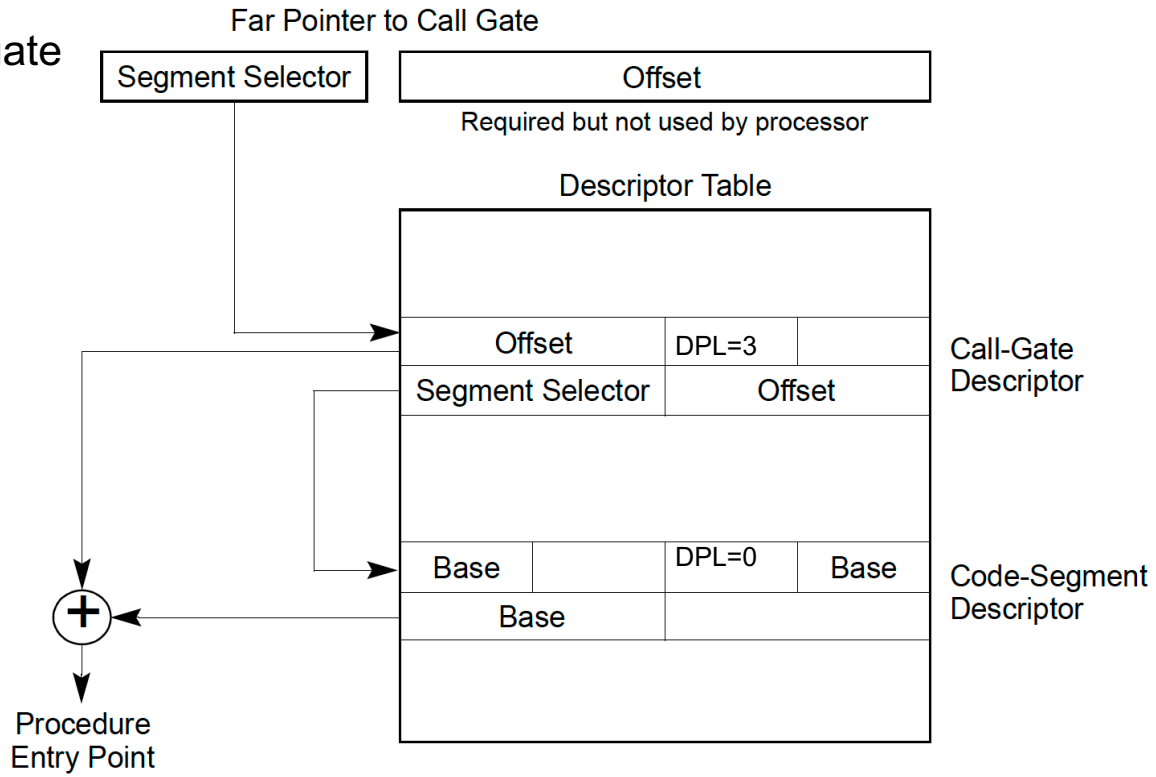
System Call Mechanisms in x86

Heinrich Heine
Universität Düsseldorf

- Call Gate
  - Descriptor in the GDT
  - Access via a `call` or `jmp` instruction
  - Stack is switched via the TSS (see lesson 8)
- Interrupt Trap Gate
  - Descriptor in the IDT
  - Access via a `int` instruction
  - Stack is switched via the TSS (see lesson 8)
- Fast System Calls
  - Configured through MSRs (Model Specific Register)
  - Access via a `syscall/sysret` instruction
  - Stack must be switched manually by the operating system

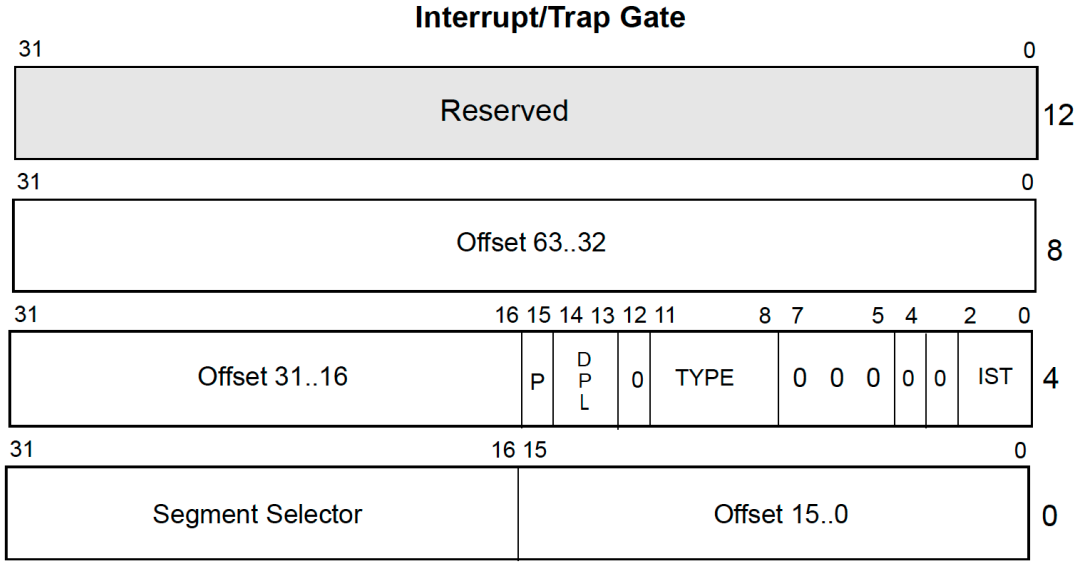
Call Gate

- Far call = calls using a call gate
- Offset in register and code segment descriptor are ignored in long mode



Interrupt/Trap Gate

- Access via `int` instruction
  - Linux uses: `int 0x80`
  - References a *Trap Gate* with index 0x80 in the IDT
- Offset = Handler address
- Segment selector chooses a segment descriptor in GDT



DPL	Descriptor Privilege Level
Offset	Offset to procedure entry point
P	Segment Present flag
Selector	Segment Selector for destination code segment
IST	Interrupt Stack Table

Interrupt/Trap Gate

Heinrich Heine
Universität Düsseldorf

- Interrupt Gate
  - *Interrupt Enable* flag is deleted
  - Used for interrupt handlers → sequential execution (on single core CPUs)
- Trap Gate
  - *Interrupt Enable* flag is not deleted
  - Used for system calls (may be interrupted by an external interrupt)
  - Used in Linux, Microsoft Windows and HeineOS

Interrupts

- External/Hardware interrupts
  - Triggered by a device (e.g., timer)
  - Come from the outside (from the CPU's point of view)

- Internal/Software interrupts
  - Triggered by the CPU itself
    - Exceptions (see next slides)
    - Assembly instruction `INT <nr>`
      - Used for *System Calls*
      - As well as debuggers setting breakpoints (int 3)

Exceptions

Heinrich Heine
Universität Düsseldorf

- **Fault**
  - Operating system can handle and recover from faults (e.g., Page Fault)
  - CPU state and the address of the fault triggering instruction are stored
- **Trap**
  - Triggered by a special instruction (e.g., INT 3 - Breakpoint)
  - Program can be continued
- **Abort**
  - Unrecoverable error
  - Triggers are hard to localize exactly
  - Cause a reboot (e.g., Double Fault)

Vectors

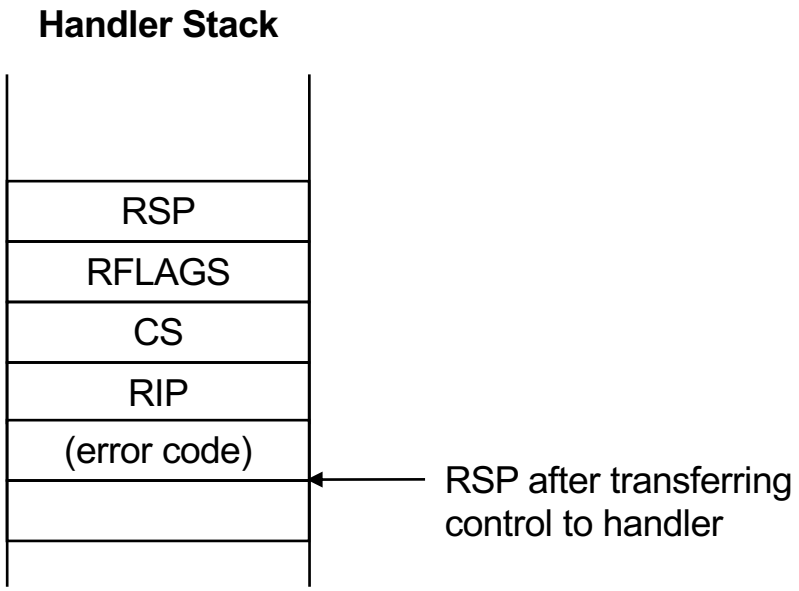
- Interrupts and exceptions are identified via a *Vector Number*
- 0 – 31 are reserved for exceptions
- 32 – 255 are freely available

- Examples for exceptions:

Vector	Exception	Vector	Exception
0	Division by 0	11	Segment not present
1	Debug	12	Stack overflow
2	NMI	13	General protection fault
3	Break	14	Page fault
4	Overflow	16	-
5	Bounds range exceeded	18	Machine check
6	Illegal instruction	...	. . .
8	Double fault		

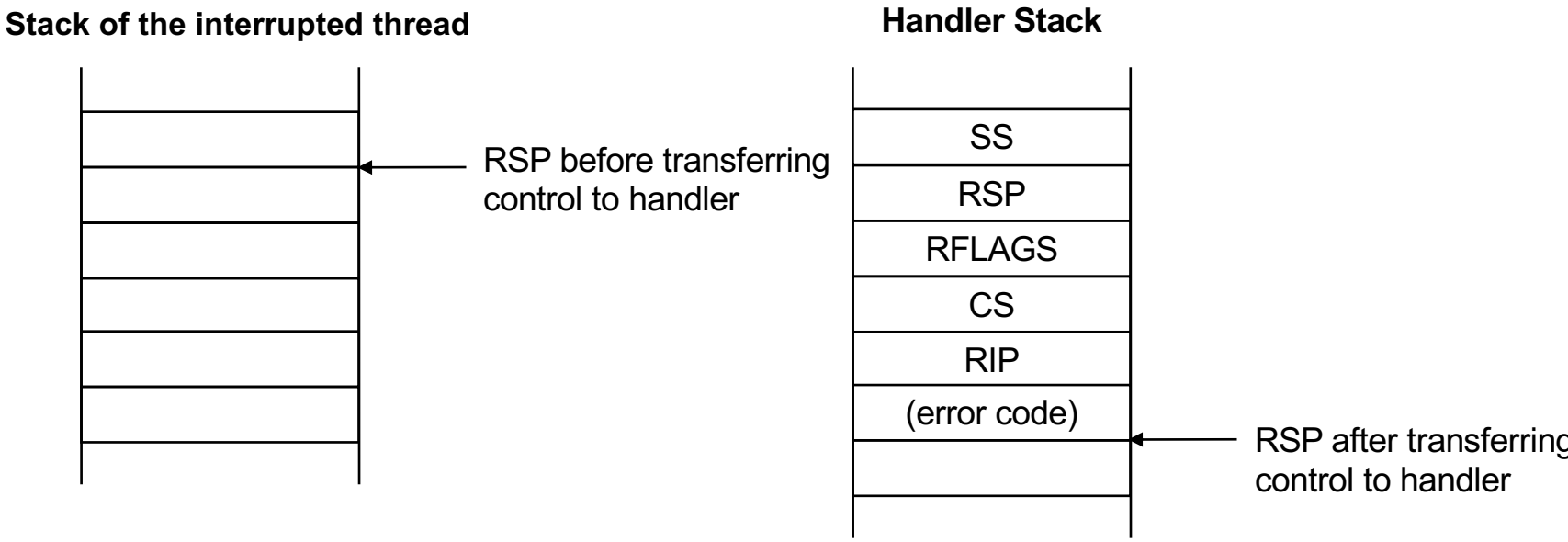
Stack Frame without Ring Change

- If the privilege level is not changed, the CPU does not switch the stack
- The interrupt handler operates on the stack of the interrupted thread
- Only some exceptions push an error code



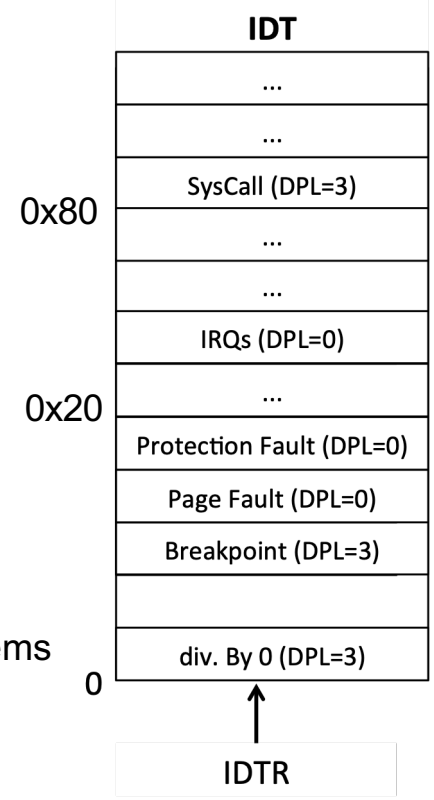
Stack Frame with Ring Change

- Kernel stack is switched by setting `rsp0` read from *Task State Segment* (see lesson 8)

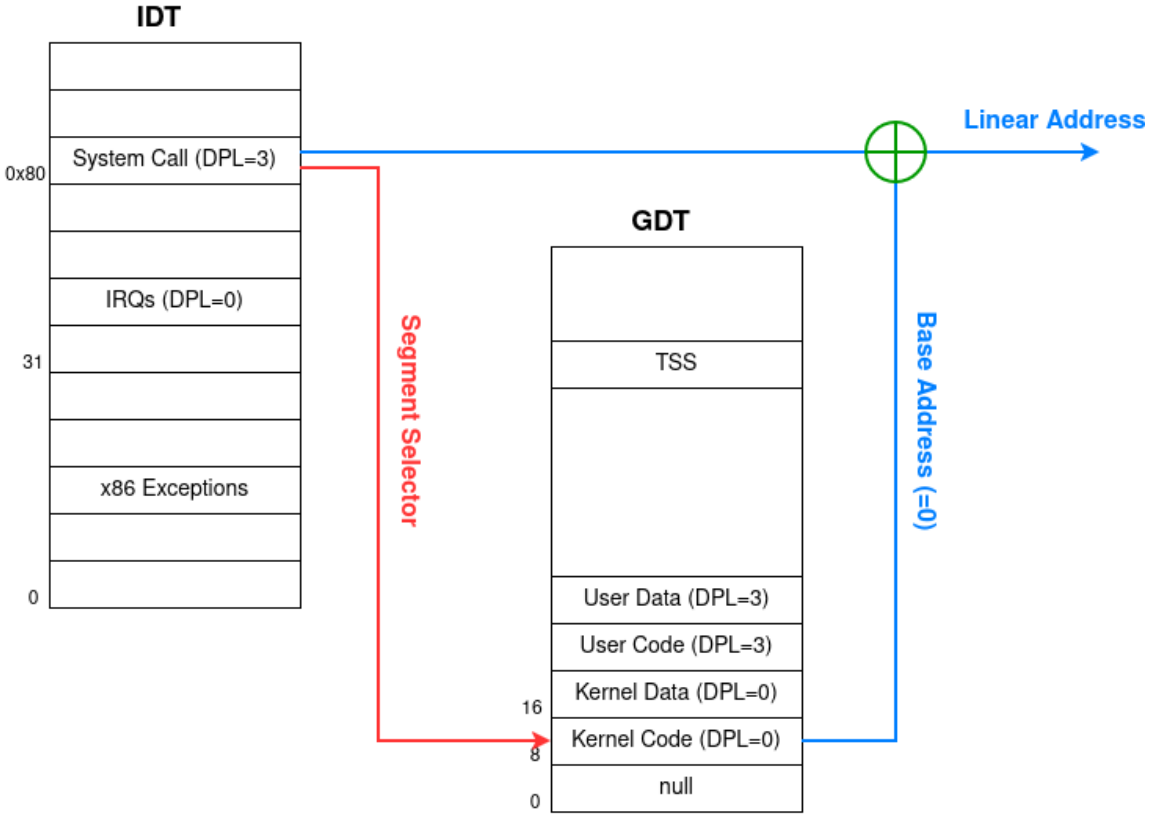


IDT: Typical Example

- Entries 0-31 are reserved for x86 CPU exceptions
  - Either DPL=0 or DPL=3, depending on use case
- Entries 32-47 for external interrupts
  - IRQs have DPL=0
  - IRQ 0 != Vector 0
- System calls via trap gate (entry 0x80)
  - DPL=3
  - Replaced by SYSENTER/SYSEXIT in modern operatings systems → faster, avoids interrupt overhead



System Call via Trap Gate



Fast System Calls: `syscall, sysret`

Heinrich Heine Universität Düsseldorf

- Use Model Specific Registers (MSRs) →
- Require specific GDT layout
- Stack muste be switched manually in syscall handler

- Use FS und GS registers
  - Only base address is used
  - Typically the GS base address points to the kernel stack

